

P2TS: A Preemptive Approach for Priority-Aware Task Scheduling in Computing Power Networks

Tao Huang^{ID}, Senior Member, IEEE, Haoxiang Qiu^{ID}, Qinqin Tang^{ID}, Li Feng^{ID}, Member, IEEE, Renchao Xie^{ID}, Senior Member, IEEE, Tianjiao Chen^{ID}, and Zehui Xiong^{ID}, Member, IEEE

Abstract—As an emerging computing paradigm, Computing Power Networks (CPNs) are dedicated to coordinating and managing network resources and computing resources to achieve interconnectivity in computing power perception. Efficient collaborative computing of massive data can be achieved through the scheduling function of CPNs. However, existing scheduling research mainly focuses on selecting network links and computing nodes, lacking consideration for task execution after scheduling, which may degrade the Quality of Service (QoS), leading to widespread failures and significant losses. To address this issue, we design a priority-aware preemptive task scheduling (P2TS) strategy for CPNs to jointly optimize task scheduling and execution in terms of success rate, average processing delay, and load balancing. Specifically, at the execution level, we propose a priority-aware preemptive mechanism (P2M) to optimize post-scheduling task execution. Then, at the scheduling level, we apply deep reinforcement learning (DRL) to optimize the scheduling process supporting the P2M in CPNs. A series of simulations are conducted to demonstrate the superiority of our strategy.

Index Terms—Computing power networks, task scheduling, priority planning, preemptive mechanism.

I. INTRODUCTION

WITH the advent of the sixth-generation (6 G) era and the rapid development of new applications such as Digital Twins (DT) [1], Autonomous Driving [2], Virtual Reality (VR) [3], and Artificial Intelligence-Generated Content (AIGC) [4], global data volume have been experiencing explosive growth. Consequently, the demand for computing power has surged. However, due to the bottleneck imposed by Moore's

Law [5], the development of computing power in a single device cannot keep pace with the data processing demands. Edge computing addresses this issue by pushing computing power to the network's edge, thereby alleviating constraints in terminal capacity and reducing delay associated with long-distance cloud transmissions through the collaborative efforts of edge nodes [6]. This results in a three-tier computing power architecture consisting of cloud, edge, and terminal levels.

Despite these advancements, the current three-tier computing architecture suffers from the isolation of dispersed and heterogeneous computing resources, leading to inefficiencies and suboptimal resource utilization. This fragmentation hinders large-scale, cross-network transmission and collaborative processing of massive datasets. The advent of Computing Power Networks (CPNs) [7], [8] offers a promising solution to these challenges by leveraging network connections to manage dispersed and heterogeneous computing resources. CPNs facilitate deep integration of computing power and networks, creating an efficient ecosystem for resource management.

In the context of CPNs, task scheduling is critical, as it directly impacts Quality of Service (QoS) and resource utilization. Traditional research on task scheduling in cloud [9] and edge computing [10], [11] primarily focuses on optimizing the matching of tasks with computing nodes, while neglecting network scheduling and control, rendering these traditional approaches unsuitable for CPNs. Recent studies in CPNs have expanded to include network links [12], task intentions [13], and application characteristics [14], proposing multi-objective combinatorial optimization strategies. These strategies ensure tasks are directed to the most suitable nodes via optimal paths, enhancing service quality and maximizing resource utilization. However, existing research predominantly concentrates on optimizing task scheduling, with inadequate attention to the arrangement and optimization of task execution once tasks reach their destination node. This oversight can lead to a significant number of delay-sensitive computing tasks missing their optimal execution time due to queuing, resulting in substantial security and economic losses, and causing a considerable disparity between expected and actual rewards, especially in high-concurrency CPNs. Recently, some researchers have noticed this issue and proposed preemptive scheduling schemes in cloud computing scenarios, achieving promising results [15].

However, compared to cloud computing, CPNs demand higher real-time performance and reliability. The existing preemptive scheduling scheme in cloud computing is unsuitable

Received 6 May 2025; revised 21 August 2025; accepted 30 August 2025. Date of publication 5 September 2025; date of current version 9 January 2026. This work was supported by the National Natural Science Foundation of China under Grant 92367104. Recommended for acceptance by X. Yu. (Corresponding author: Renchao Xie.)

Tao Huang and Renchao Xie are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: htiao@bupt.edu.cn; Renchao_xie@bupt.edu.cn).

Haoxiang Qiu, Qinqin Tang, and Li Feng are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: haoxiangqiu@bupt.edu.cn; qqtang@bupt.edu.cn; fengli@bupt.edu.cn).

Tianjiao Chen is with China Mobile Research Institute, Beijing 100053, China (e-mail: chentianjiao@chinamobile.com).

Zehui Xiong is with the School of Electronics, Electrical Engineering and Computer Science (EEECS), Queen's University Belfast, BT7 1NN Belfast, U.K. (e-mail: z.xiong@qub.ac.uk).

Digital Object Identifier 10.1109/TMC.2025.3606454

for CPNs because of its relatively simple preemptive mechanism and a single evaluation indicator. Therefore, to jointly optimize task scheduling and execution in terms of success rate, processing delay, and load balancing in CPNs, we propose a priority-aware preemptive task scheduling (P2TS) strategy for CPNs and design a P2 twin delayed deep deterministic policy gradient (P2TD3) algorithm for implementation. Specifically, our contributions are as follows.

- We consider a CPN system that integrates both computing and networking resources. Based on this, we comprehensively analyze optimization opportunities for tasks at the execution and scheduling levels and propose a P2TS strategy tailored for CPNs.
- At the execution level, we propose a priority-aware preemptive mechanism (P2M). Specifically, we meticulously define task priorities across multiple dimensions and develop a strategy for task execution order. Based on these priorities, a robust preemptive mechanism is designed to ensure that higher-priority tasks can effectively preempt computing resources when needed.
- At the scheduling level, given the high concurrency dynamics of CPNs, we apply deep reinforcement learning (DRL) to formulate the scheduling problem supporting the P2M in CPNs and design a P2TD3 algorithm to solve it.
- Through extensive simulations, we evaluate the performance of the proposed P2TD3 algorithm against four baseline algorithms. The results demonstrate the superior performance and efficiency of the proposed strategy.

The rest of this paper is organized as follows. Section II reviews related work about CPNs, task scheduling, and task preemptive execution. Section III describes the CPNs scheduling system under P2TS. Section IV introduces the P2M within P2TS and analyzes related delay. Section V uses DRL to formulate the scheduling problem under P2TS in CPNs and design the P2TD3 algorithm to solve it. Simulation results are discussed in Section VI, with the conclusion in Section VII.

II. RELATED WORK

This section briefly reviews related work about computing power networks, task scheduling, and task preemptive execution. Then, we highlight the distinctions between this paper and previous research.

A. Computing Power Networks

The concept of Computing Power Networks, introduced in 2019, has significantly attracted research attention. Initial work by [16] explored CPNs through cloud, network, and edge collaboration. The International Telecommunication Union (ITU) formalized CPNs in the ITU-T Y.2501 recommendation [17], detailing its tiered architecture. Subsequent research by [18] examined integrating CPNs with 6 G services, covering computing, storage, and network deployment. Zhang et al. proposed Segment Identifier as a Service (SIDaaS) based on Segment Routing over IPv6 (SRv6), along with a Programmable Service System (PSS) that enhances business orchestration by integrating computing power and networks [19]. Sun et al. introduced a

wireless CPN (WCPN) to orchestrate heterogeneous nodes for task completion [20]. Tang et al. integrated intent networks into CPNs to optimize task intent and resource matching [13]. These studies on CPNs have significantly accelerated their evolution and development, providing a solid foundation for the practical and mature application of CPNs.

B. Task Scheduling

Optimized scheduling schemes and algorithms are crucial for system performance and have been extensively studied. Various scheduling schemes have been proposed for cloud computing [21], [22], [23], [24], [25], edge computing [26], [27], [28], [29], [30], [31], and cloud-edge computing [32], targeting optimization in delay and cost, and energy consumption. However, traditional studies focus on matching tasks with virtual machines or small-scale nodes, neglecting network integration and large-scale collaboration. In CPNs, Xie et al. proposed an edge computing framework with a DRL-based task scheduling strategy for load balancing [33]. Tang et al. developed a Service Intent-aware Task Scheduling (SIaTS) framework and an auction-based algorithm for optimal resource matching [13]. Feng et al. introduced collaborative optimization of nodes and links to reduce task delay, using Lyapunov control theory and DRL for load balancing [12]. Chen et al. proposed a two-stage evolutionary search framework to optimize task offloading in edge CPNs [34], focusing on improving task execution efficiency and resource utilization. These studies examined task scheduling strategies in CPNs from diverse perspectives, addressing the optimization of task scheduling to meet various objectives across different scenarios. However, these existing works do not pay enough attention to optimizing task execution after scheduling, which may lead to huge differences between ideal and actual benefits in scheduling strategies. This paper is aimed at studying this issue.

C. Task Preemptive Execution

Task preemptive execution, where urgent tasks interrupt ongoing tasks for prioritized execution, is underexplored. Cheng et al. proposed a preemptive mechanism in cloud computing based on remaining response time alone, which lacks assessing preemption necessity globally and may lead to unnecessary interruptions [15]. When studying the delay-prioritized and reliable task scheduling in CPNs, Xie et al. allowed newly scheduled emergency tasks to preempt the last regular task in the computing queue when the queue reaches full capacity [35]. This approach ensures that a portion of the emergency tasks can successfully enter the computing queue. However, the scope of this preemption strategy is limited, as it only addresses the preemption of a single task and only ensures the timely entry into the computing queue of partial emergency tasks. Additionally, it lacks a strategy for managing preempted tasks. Other studies claiming preemptive scheduling often focus on resource partitioning [36] by task weights without true preemption. This paper aims to develop a comprehensive preemptive mechanism for optimizing task execution in CPNs.

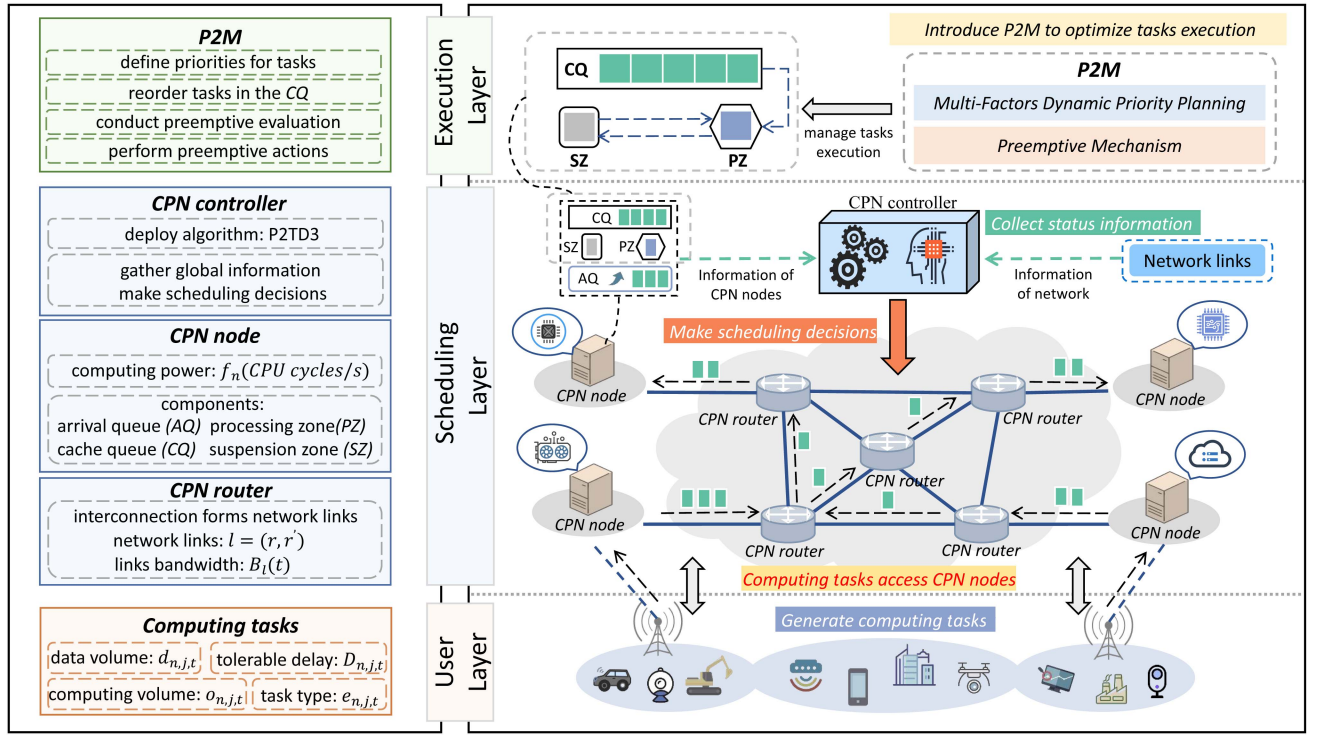


Fig. 1. Illustration of the CPN system under the proposed P2TS.

In summary, existing research has provided rich theoretical and practical foundations for this study. However, notable gaps remain in optimizing post-scheduling task execution and developing comprehensive task preemption mechanisms. Therefore, this paper focuses on addressing these critical limitations, aiming to propose innovative solutions that enhance the efficiency and effectiveness of task execution in CPNs.

III. SYSTEM DESCRIPTION

In this section, we first introduce the CPN system for P2TS. Subsequently, we describe the workflow of the system under the P2TS strategy.

In the CPN system considered, the complete lifecycle of computing tasks can be broadly categorized into three primary stages: generation, scheduling, and execution. To efficiently manage computing tasks, the CPN system is logically organized into three layers: the user layer, the scheduling layer, and the execution layer. These layers are described in detail as follows.

- **User Layer:** Driven by diverse computing and network interaction demands, entities such as individuals, communities, and organizations continuously generate computing tasks through various terminals, such as autonomous driving, intelligent model training, AIGC, and VR. Limited by terminal computing power, these tasks can be offloaded to CPN nodes for processing.
- **Scheduling Layer:** This layer is primarily responsible for matching computing tasks to appropriate CPN nodes, determining where each task should ultimately be executed. The scheduling process involves coordinating computing

power and network resource availability, with decisions made by a centralized controller. Once determined, tasks are forwarded to the designated CPN nodes for execution.

- **Execution Layer:** The execution layer is focused on optimizing the execution of computing tasks. Upon receiving the tasks from the scheduling layer, the relevant optimization mechanisms are activated at the destination CPN nodes to further coordinate and manage the execution process.

Based on the aforementioned description, the following sections will provide a detailed introduction to our CPN model.

A. CPN System Model

As illustrated in Fig. 1, the CPN system consists of heterogeneous CPN nodes ($\mathcal{N} = \{1, \dots, N\}$), multiple CPN routers ($\mathcal{R} = \{1, \dots, R\}$), and a CPN controller. Time is divided into equal interval scheduling slots with a duration ΔT , represented as $\mathcal{T} = \{1, \dots, T\}$.

CPN Nodes: Each heterogeneous CPN node $n \in \mathcal{N}$ maintains several components, including an arrival queue (AQ), a cache queue (CQ), a processing zone (PZ), and a suspension zone (SZ). They are detailed as follows.

- **AQ:** At the beginning of each time slot, new tasks arrive randomly at the CPN nodes. These tasks enter the AQ in sequence, waiting for scheduling to the destination CPN nodes.
- **CQ:** After the tasks are forwarded to their destination CPN nodes, they enter the CQ, where they await entry into the PZ. In the CQ, tasks are arranged according to a proposed

multi-factor dynamic priority planning scheme, which is discussed in detail in Section IV.

- *PZ*: The zone is responsible for task computing, capable of computing one task at a time in computing power f_n (cycles/s) of the CPN node n . Tasks in the *CQ* will enter the *PZ* in sequence. If a task is currently being computed, subsequent tasks must wait until its completion before entering the *PZ*.
- *SZ*: The zone holds the task that was initially computed in the *PZ* but was preempted by higher-priority tasks. Task in the *SZ* may have the chance to re-enter the *PZ* to continue its computing after the higher-priority task in the *PZ* is completed. To avoid backlog, *SZ* is limited to holding one task at a time.

Network Links: Let $\mathcal{L} = \{1, \dots, L\}$ represent the set of network links. Each link $l = (r, r')$ connects two CPN routers r and r' ($r, r' \in \mathcal{R}$) and has a real-time bandwidth $B_l(t)$ during time slot t . The weight of link l is defined as $\frac{1}{B_l(t)}$. To determine the shortest path between any two CPN nodes n and m , we apply the Floyd algorithm, which results in a path $P_{n,m}(t)$, serving as the default forwarding path. Here, the shortest path refers to the path that enables the fastest task transmission under the prevailing network conditions, which is deeply related to the real-time bandwidth of the network links. However, the bandwidth of network links may fluctuate due to various factors, and the Floyd algorithm inherently lacks mechanisms to avoid congestion or adapt to dynamic network conditions. To mitigate network congestion, we dynamically implement the Floyd algorithm, recalculating the shortest paths for each time slot based on the real-time bandwidth of network links rather than calculating them only once and reusing them for the entire scheduling period. Specifically, at the beginning of each time slot, the CPN router at one end of a link l sends a bandwidth probe packet to the CPN router at the other end, which will respond with an acknowledgment. This exchange allows the CPN routers to obtain the real-time bandwidth $B_l(t)$ of l , and the weight of link l is updated as $\frac{1}{B_l(t)}$. Subsequently, the Floyd algorithm is employed to compute the shortest paths between CPN nodes for that time slot, which are then used in the task scheduling process. It is important to emphasize that these shortest paths are valid only within the specific time slot in which they are calculated. At the beginning of each subsequent time slot, the paths are recalculated based on the updated real-time bandwidth information, following the same procedure mentioned above.

Even if network links experience congestion during transmission, routers can detect and feedback the real-time bandwidth information to the controller at the beginning of the next time slot. Based on this updated bandwidth, the Floyd algorithm recalculates the optimal shortest path for the new time slot. For the congested link, its bandwidth decreases dramatically and its link weight becomes large accordingly, which will be excluded by the Floyd algorithm. As a result, the link will not participate in the transmission of tasks and can not affect the scheduling process at the new time slot. This time-slot-based link bandwidth detection and dynamic recalculation of shortest paths effectively prevent tasks from traversing congested paths. Additionally, it provides a “buffer period” for congested link, allowing it to

gradually transition from congestion to smoothness, preparing for possible task transmission in subsequent time slots.

Computing Tasks: Let $\varepsilon_n(t) = \{\varepsilon_{n,1}(t), \dots, \varepsilon_{n,J}(t)\}$ denote the set of tasks newly accessing the CPN node n at the beginning of time slot t . As the j -th newly arrived task, $\varepsilon_{n,j}(t)$ can be described with a tuple $(d_{n,j,t}, o_{n,j,t}, e_{n,j,t}, D_{n,j,t}^{tlr})$, denoting the data volume (in bits), computing volume (in CPU cycles), task type (e.g., autonomous driving, remote surgery, AIGC), and tolerable delay, respectively. If $\varepsilon_{n,j}(t)$ is to be scheduled to the CPN node m for execution, its scheduling decision is $a_{n,j}^m(t) = 1$; otherwise, $a_{n,j}^m(t) = 0$.

CPN Controller: The CPN controller integrates a task scheduling module internally, which deploys a trained task scheduling algorithm. As the “brain” of the CPN system, the controller takes the responsibility of gathering global information, making scheduling decisions, and continuously optimizing and further improving the scheduling algorithm based on system feedback during the continuous scheduling process.

Before the beginning of each time slot, the CPN controller broadcasts a lightweight control signal to all CPN nodes and CPN routers within the system, notifying them to collect and upload the required information at the beginning of the upcoming time slot. The control signal adopts a send-confirm-retry mechanism to mitigate the risk of signal failure caused by transient network fluctuations. Based on the control signal, at the beginning of each time slot, each CPN node summarizes relevant information about newly arrived tasks in the *AQ*, queuing tasks in the *CQ*, task being computed in the *PZ*, and task being suspended in the *SZ*, including tolerable delay, computing volume, task type, and other key parameters. Then, the *AQ*, *CQ*, *PZ*, and *SZ* data packets are generated and sent to the CPN controller. In parallel, the CPN routers encapsulate real-time links bandwidth information for the current time slot into bandwidth data packets, which are also sent to the CPN controller. The CPN controller parses the incoming packets and aggregates the information into a global state table to form a comprehensive and time-slot-based view of both tasks and network. Based on this table, the CPN controller constructs a network weight view using the link bandwidth information and applies the Floyd algorithm to compute the default transmission paths between CPN nodes for the current time slot. Simultaneously, the task-related information in the table is input to the task scheduling module, where the trained task scheduling algorithm outputs the optimal scheduling decisions for the tasks in the current time slot. Finally, the CPN controller issues both the computed transmission paths and the scheduling decisions to the relevant CPN routers and CPN nodes, thereby guiding tasks to be transmitted to their designated destination nodes along the predetermined paths.

A single centralized CPN controller may cause some issues, particularly the single-point failure. To address these potential challenges, we propose several safeguard mechanisms. First, we introduce a standby CPN controller to mitigate the potential single-point failure risk. The standby controller periodically sends “heartbeat detection packets” at short time intervals to verify the health of the primary controller. If no response is received from the primary controller within a preset time window, the failure is detected, and the standby controller takes

over the primary controller's work. It then broadcasts the failure notification to all CPN nodes and routers, prompting them to send their relevant data to the standby controller and follow its task scheduling decisions, ensuring continuity and minimizing disruptions. With advancements in hardware, computing power, and algorithms, a single centralized CPN controller can effectively manage a CPN system within a certain scale. However, as the system expands into ultra-large-scale CPNs, scalability issues may arise. In this case, the large-scale CPN can be divided into several regions. Within each region, deployment follows our existing approach, with a centralized CPN controller managing task scheduling. Inspired by work [37], we propose introducing a main CPN controller to manage information sharing and task collaboration across regions. When a new region is added, its CPN controller can be connected to the main controller to enable information sharing and task collaboration with other regions.

B. System Workflow Under P2TS

1) *Computing Task Accessing CPNs*: The computing tasks generated by the user layer are encapsulated into data packets, which are transmitted to the nearest base stations via wireless channels and subsequently routed to the CPNs. Upon accessing the CPN nodes, the tasks enter the *AQ* to await scheduling. To mitigate interference among concurrent transmissions, an advanced modulation technique known as orthogonal frequency division multiplexing (OFDM) is employed. This technique allocates distinct sub-channels for each computing task, ensuring that each sub-channel is assigned a unique frequency band orthogonal to others, thereby minimizing cross-interference. The total available bandwidth is evenly divided across the sub-channels. The wireless transmission rate for a task $\varepsilon_{n,j}(t)$ accessing CPN node n can be derived using Shannon's capacity formula:

$$v_{n,j,t} = \frac{B_n^{tol}}{|\varepsilon_n(t)|} \cdot \log_2 \left\{ 1 + \frac{E_{n,j}(t) \cdot G_{n,j}(t)}{|\varepsilon_n(t)| \cdot \sigma_{n,j}^2(t)} \right\}, \quad (1)$$

where B_n^{tol} is the total bandwidth of the channel, $E_{n,j}(t)$ is the transmission power, $G_{n,j}(t)$ is the channel gain, and $\sigma_{n,j}^2(t)$ represents the Gaussian white noise power. The wireless transmission delay for task $\varepsilon_{n,j}(t)$ is given by:

$$D_{\varepsilon_{n,j}(t)}^{wtran}(t) = \frac{d_{n,j,t}}{v_{n,j,t}}. \quad (2)$$

2) *Computing Task Scheduling*: For tasks in the *AQ*, we adopt a centralized control approach to achieve optimal matching with CPN nodes. The CPN controller first collects status information, including the current states of CPN nodes, network links, and newly arrived tasks. It then applies the Floyd algorithm to compute the shortest path between CPN nodes. Based on these shortest paths, a default forwarding directory is created and disseminated to the CPN nodes and routers. The CPN controller analyzes the system's current state and issues task scheduling decisions to each CPN node. For a task with $a_{n,j}^m(t) = 1 (n = m)$, it does not require forwarding and directly enters the *CQ* and waits for computing; for a task with $a_{n,j}^m(t) = 1 (n \neq m)$, it must be forwarded from CPN node n to node m along with the default

path. The computing delay for task $\varepsilon_{n,j}(t)$ with $a_{n,j}^m(t) = 1$ can be expressed as:

$$D_{\varepsilon_{n,j}(t)}^{comp}(t) = \frac{o_{n,j,t}}{f_m}. \quad (3)$$

3) *Computing Task Execution*: Once tasks are forwarded to their respective destination CPN nodes, they enter the *CQ* to await processing. At this stage, the P2M mechanism is activated to manage the execution process of tasks, which will be described in detail in Section IV.

IV. PRIORITY-AWARE PREEMPTIVE MECHANISM

In this section, we first provide a detailed introduction to the P2M, consisting of two multi-factor dynamic priority planning and the preemptive mechanism. Then, we discuss the related delays associated with computing tasks under the P2M.

A. Multi-Factors Dynamic Priority Planning

In high-concurrency CPNs, scheduling decisions may result in a large number of tasks accumulating in the *CQ* awaiting computing. If tasks are processed in an unordered manner, it could lead to inefficiency and confusion. The traditional first-come, first-served (FCFS) strategy fails to account for the urgency of task execution, thus hindering the effective utilization of computing resources for prioritizing urgent tasks. To address this, we propose a multi-factor dynamic priority planning strategy, which is applied post-scheduling to determine a rational execution order for tasks within the *PZ*, *SZ*, and *CQ*. The priority of tasks, reflecting their execution urgency, also serves as the foundation for the preemptive mechanism.

Given the limited computing resources and the high concurrency of tasks, partial tasks may not be completed within the current time slot, even after being scheduled to their destination nodes. These tasks will continue to be executed in subsequent time slots as legacy tasks. Consequently, tasks that exist in the system in time slot t may have been generated in either the current or a previous time slot, but they remain incomplete. To distinguish between the time slot of task generation and the current time slot, we denote the task generation time slot as $\tau (\tau \in \mathcal{T})$ and the current time slot as $t (t \in \mathcal{T})$, with $\tau \leq t$. The priority of task $\varepsilon_{n,j}(t)$ is aware and defined based on the following factors:

- *Task Type*. The computing tasks in CPNs may vary in type, and their importance can differ accordingly. For example, tasks related to autonomous driving, remote surgery, or air traffic control are of critical importance. A failure in these tasks could lead to severe accidents and significant harm to individuals and society. In contrast, tasks such as routine data processing or background computations carry less immediate risk, and their failure would not lead to severe or immediate consequences. Therefore, this factor is quantified by $e_{n,j,\tau} \in [1, 10]^+$, where higher values indicate greater importance and higher potential social loss in the event of failure.
- *Delay Elasticity*. In CPNs, computing tasks with lower tolerable delay require more timely computing to avoid

timeout, while tasks with higher tolerable delay offer more flexibility and do not need immediate computing. To quantify and assess the demand for timely computing from the perspective of tolerable delay, we introduce delay elasticity, which is defined by:

$$s_{n,j,\tau} = \frac{1}{D_{n,j,\tau}^{ttr}}, \quad (4)$$

where a higher value indicates that less buffer time remains for the task to meet its deadline, necessitating earlier computing.

- *Relative Execution Speed.* Those computing tasks with smaller computing volume can be completed more rapidly than those with larger volume, thereby releasing computing resources more quickly and enabling faster processing of subsequent tasks. To quantify this aspect, we define relative execution speed, expressed as:

$$g_{n,j,\tau} = \frac{1}{o_{n,j,\tau}}, \quad (5)$$

where a higher value indicates that the task can be completed more rapidly with the same computing resources, leading to a quicker release of resources and a reduction in the average queuing delay of tasks within the *CQ*.

- *Task Lifespan Ratio.* The task type, delay elasticity, and relative execution speed are all determined at the time of task generation. When the priority defined by these factors is very low, certain tasks may not be processed and time out after waiting for a long period, which is unfair. We hope to improve their chances of being processed as their waiting time increase. Therefore, the task lifespan ratio is introduced and defined as:

$$u_{n,j,\tau}(t) = \frac{t - \tau}{D_{n,j,\tau}^{ttr}}. \quad (6)$$

This ratio reflects the elapsed time relative to the task's tolerable delay. A ratio close to 1 indicates that the task is approaching its deadline, thereby increasing its urgency. For those long-waiting and low-priority tasks, this ratio provides an opportunity to improve their priorities.

Based on the above factors, the execution priority for task $\varepsilon_{n,j}(\tau)$ at time slot t is defined as:

$$p_{n,j,\tau}(t) = \omega_1 e_{n,j,\tau} + \omega_2 s_{n,j,\tau} + \omega_3 g_{n,j,\tau} + \omega_4 u_{n,j,\tau}(t), \quad (7)$$

where $\omega_1, \omega_2, \omega_3$ and ω_4 are weight factors satisfying:

$$\omega_1 + \omega_2 + \omega_3 + \omega_4 = 1. \quad (8)$$

As for the determination of specific values for these weight factors, one or more values can be set higher based on the desired goals and specific scenarios, ensuring that their corresponding priority factors become dominant.

Once the priorities are determined, tasks in the *CQ* are re-ordered from highest to lowest priority to ensure that they enter the *PZ* in an efficient and rational sequence for execution. We denote $K_{n,j,\tau}^m(t) \in \mathbb{Z}^+$ as the unique sequence number for task $\varepsilon_{n,j}(\tau)$ in the *CQ* of CPN node m at time slot t . Specifically,

tasks initially in the *SZ* are assigned $K_{n,j,\tau}^m(t) = -1$, while tasks initially in the *PZ* are assigned $K_{n,j,\tau}^m(t) = -2$ at time slot t .

B. Preemptive Mechanism

While the execution order of tasks is determined by their priorities, a phenomenon known as “forced waiting” is inevitable, whereby newly arrived high-priority tasks must wait for the current task in the *PZ* to be completed, causing higher delay and the potential risk of timeouts. To mitigate this issue, a preemptive mechanism is introduced, allowing eligible tasks to directly enter the *PZ* and preempt computing resources for immediate execution. This action forces the current task in the *PZ* (*PZ* task) to be interrupted and suspended or delays the task in the *SZ* (*SZ* task) entering the *PZ*.

However, if priority is used as the basis for preemptive behavior alone, it may lead to suboptimal outcomes. For example, consider the scenario where task A is being executed in the *PZ* and a higher-priority task B arrives. If task A's left delay is short and it cannot tolerate the execution of task B before it, whereas task B can tolerate the execution of task A before it. Despite this, task A will be preempted due to its lower priority, which results in task A being interrupted and suspended until task B is completed. This could lead to task A's failure. Therefore, relying solely on priority for preemptive behavior may cause unnecessary interruptions, decrease the overall success rate, and waste computing resources. In response, a series of preemptive evaluations will be conducted to manage the preemptive behavior for time slot t , which is detailed in Algorithm 1.

Before the preemptive evaluation, we mark the tasks that potentially be preempted (*PZ* or *SZ* task) as target task, and the tasks that might take preemptive action (higher priority tasks in the *CQ*) as uncertain tasks. After the preemptive evaluation, eligible uncertain tasks are designated as preemptive tasks, while others are classified as regular tasks. Once task $K_{n,j,\tau}^m(t)$ is marked as a preemptive task, A flag $\theta_{n,j,\tau}^m(t) = 1$ is set for it. Otherwise, $\theta_{n,j,\tau}^m(t) = 0$. The specific preemptive evaluation will involve several relevant delay concepts of task $K_{n,j,\tau}^m(t)$: *i*): Computing delay $D_{K_{n,j,\tau}^m(t)}^{comp}(t)$, the delay required for computing of task $K_{n,j,\tau}^m(t)$; *ii*): Left delay $D_{K_{n,j,\tau}^m(t)}^{left}(t)$, that is, the task will timeout after $D_{K_{n,j,\tau}^m(t)}^{left}(t)$; *iii*): Remaining computing delay $D_{K_{n,j,\tau}^m(t)}^{remcomp}(t)$, which is the computing delay required for the uncompleted portion.

Preemptive evaluation is triggered only when a task is being computed in the *PZ* and there are uncertain tasks in the *CQ*. Otherwise, the tasks of the CPN node m will be executed in the following order: the *PZ* task, the *SZ* task (if any), and then the task in *CQ*. The situations that trigger preemptive evaluation can be specifically divided into two types, and we will take the CPN node m as an example, which is detailed as follows.

1) *Situation A:* There is a task -2 in the *PZ*, and the *SZ* is empty. The target task is task -2. We define the delay comparison function under *Situation A* as:

$$\Upsilon_a(x, K_{n,j,\tau}^m(t)) = x - D_{-2}^{remcomp}(t)$$

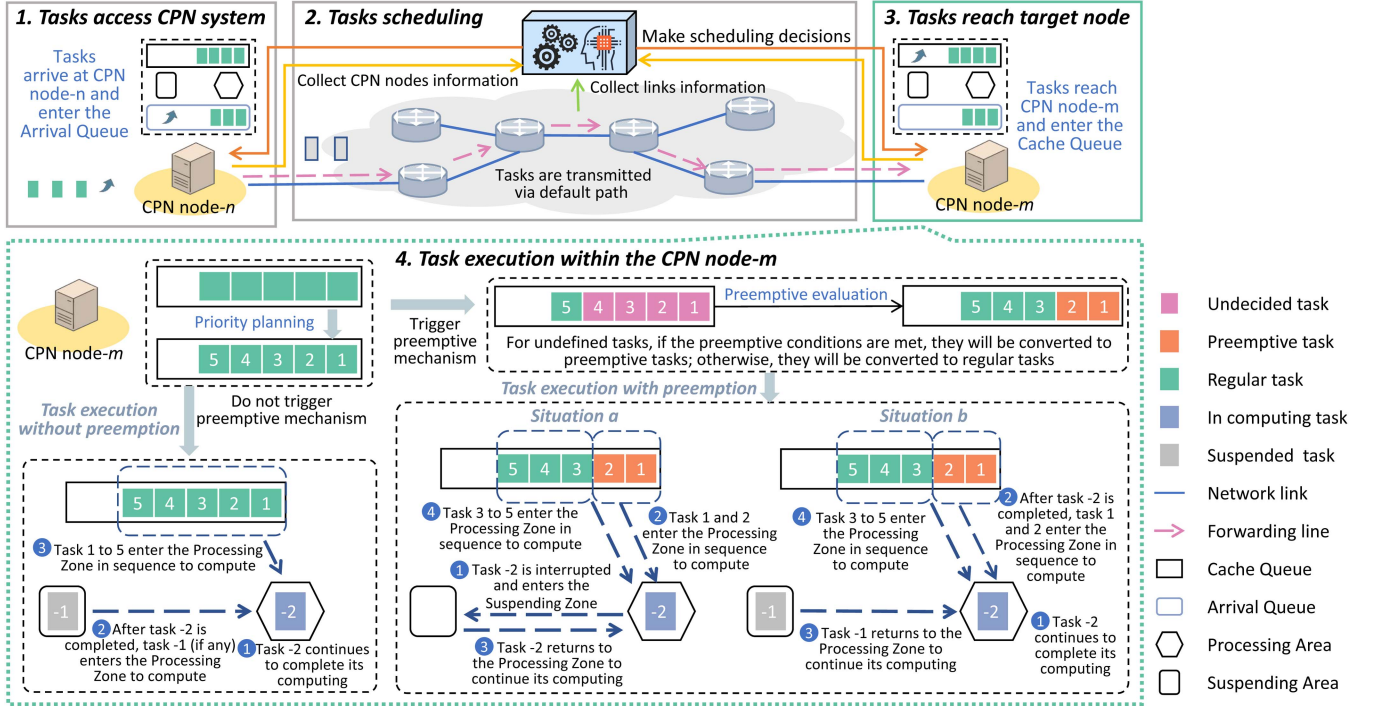


Fig. 2. Illustration of the detailed task execution process in different situations within P2TS.

$$- \sum_{K_{n',j',\tau'}^m(t)=0}^{K_{n,j,\tau}^m(t)} D_{K_{n',j',\tau'}^m(t)}^{comp}(t), \quad (9)$$

where $x \in \{D_{K_{n,j,\tau}^m(t)}^{left}(t), D_{-2}^{left}(t)\}$, $(K_{n,j,\tau}^m(t) > 0)$.

$\Upsilon_a(D_{K_{n,j,\tau}^m(t)}^{left}(t), K_{n,j,\tau}^m(t))$ uses the left delay of task $K_{n,j,\tau}^m(t)$ minus the remaining computing delay of the target task and the computing delay of the first $K_{n,j,\tau}^m(t)$ tasks in CQ to determine whether task $K_{n,j,\tau}^m(t)$ still has sufficient time to be completed when the first $K_{n,j,\tau}^m(t) - 1$ tasks in CQ and the target task are computed before it.

The preemptive evaluation is performed on the uncertain tasks in sequence: whether the uncertain task $K_{n,j,\tau}^m(t)$ satisfies $\Upsilon_a(D_{K_{n,j,\tau}^m(t)}^{left}(t), K_{n,j,\tau}^m(t)) \geq 0$, if so, it means the task $K_{n,j,\tau}^m(t)$ can tolerate the first $K_{n,j,\tau}^m(t) - 1$ tasks in the CQ and the target task being computed before it. Thus, the task $K_{n,j,\tau}^m(t)$ will not take preemptive action, and the preemptive evaluation terminates. Task $K_{n,j,\tau}^m(t)$ and its subsequent tasks become regular tasks; if it is not met, it indicates task $K_{n,j,\tau}^m(t)$ is urgent to be computed first or it will definitely fail. Then, further evaluation is conducted:

- If $\Upsilon_a(D_{-2}^{left}(t), K_{n,j,\tau}^m(t)) \geq 0$, it indicates that the target task can tolerate the first $K_{n,j,\tau}^m(t)$ tasks in CQ to be computed first. So the task $K_{n,j,\tau}^m(t)$ transform into preemptive task;
- If $\Upsilon_a(D_{-2}^{left}(t), K_{n,j,\tau}^m(t)) \leq 0$, it indicates that one of the task $K_{n,j,\tau}^m(t)$ and the target task will fail. The target task is immediately discarded to ensure the computing of the tasks with higher priority, and the preemptive evaluation

terminates. Then, all the tasks in CQ are marked as regular tasks.

As shown in Fig. 2, once the preemptive evaluation is finished, the tasks will be executed according to the following. The target task is interrupted and enters the SZ for suspending until the preemptive tasks are completed. At last, the regular tasks are executed sequentially after the target task.

2) *Situation B*: There is a task -2 in the PZ , and a task -1 in the SZ . The target task is task -1. Task -2 cannot be preempted and suspended. It will continue computing because the SZ is not empty. We then define another delay comparison function under *Situation B*:

$$\Upsilon_b(x, K_{n,j,\tau}^m(t)) = x - D_{-2}^{remcomp}(t) - D_{-1}^{remcomp}(t) - \sum_{K_{n',j',\tau'}^m(t)=0}^{K_{n,j,\tau}^m(t)} D_{K_{n',j',\tau'}^m(t)}^{comp}(t), \quad (10)$$

where $x \in \{D_{K_{n,j,\tau}^m(t)}^{left}(t), D_{-1}^{left}(t)\}$, $(K_{n,j,\tau}^m(t) > 0)$. The preemptive evaluation process is similar to *Situation A*: just notice that the target task is task -1, and the delay comparison function is $\Upsilon_b(x, K_{n,j,\tau}^m(t))$. Then, the preemptive tasks enter the PZ sequentially for execution after task -2 is completed, the target task continues to be suspended until the preemptive tasks are completed. At last, the regular tasks are executed.

The task's execution in time slot t can be determined via the preemptive evaluation, as shown in Fig. 2. Record the maximum sequence of preemptive tasks in the CPN node m for $\Phi_m(t)$. If no preemptive task, $\Phi_m(t) = 0$.

Algorithm 1: Preemptive Evaluation Algorithm.

Input: CPNs tasks related information after scheduling.
Output: The execution process in each CPN node.

```

1 for each CPN node  $m = 1, 2, \dots, N$  do
2   if the PZ is not empty then
3     if the SZ is empty then
4       Mark PZ task as target task, CQ tasks with
         higher priority than PZ task as uncertain
         tasks;
5       for each task  $K_{n,j,\tau}^m(t)$  in uncertain tasks do
6         if  $\Upsilon_a(D_{K_{n,j,\tau}^m}^{left}(t), K_{n,j,\tau}^m(t)) \geq 0$  then
7           The task  $K_{n,j,\tau}^m(t)$  and the tasks after
             it are marked as regular tasks;
8           Terminate the preemptive evaluation;
9         else
10          if  $\Upsilon_a(D_{-2}^{left}, K_{n,j,\tau}^m(t)) \geq 0$  then
11            The task  $K_{n,j,\tau}^m(t)$  is marked as
              preemptive task;
12          else
13            Directly discard the target task;
14            Terminate the preemptive
              evaluation;
15       The target task (if not be discarded) is
         suspended;
16       Preemptive tasks enter PZ first in order;
17       The target task enter PZ from SZ;
18       Regular tasks enter PZ in sequence;
19     else
20       Mark the task in SZ as target task, CQ tasks
         with higher priority than SZ task as
         uncertain tasks;
21       for each task  $K_{n,j,\tau}^m(t)$  in uncertain tasks do
22         if  $\Upsilon_b(D_{K_{n,j,\tau}^m}^{left}(t), K_{n,j,\tau}^m(t)) \geq 0$  then
23           The task  $K_{n,j,\tau}^m(t)$  and the tasks after
             it are marked as regular tasks;
24           Terminate the preemptive evaluation;
25         else
26           if  $\Upsilon_b(D_{-1}^{left}, K_{n,j,\tau}^m(t)) \geq 0$  then
27             The task  $K_{n,j,\tau}^m(t)$  is marked as
              preemptive task;
28           else
29             Directly discard the target task;
30             Terminate the preemptive
              evaluation;
31       Continue to complete the in PZ;
32       Preemptive tasks enter the PZ first in order;
33       The target task (if not be discarded) enters
         PZ;
34       Regular tasks enter PZ in turn;
35     else
36       The tasks in the CQ enter the PZ sequently;

```

C. Delay Model

The related delay about task $K_{n,j,\tau}^m(t)$ under the P2M will involve the total computing volume (denoted as $O_{K_{n,j,\tau}^m}^{all}(t)$) of the tasks that should be computed before task $K_{n,j,\tau}^m(t)$ according to the preemptive evaluation. For those tasks in the

CQ, the $O_{K_{n,j,\tau}^m}^{all}(t)$ can be expressed as follows.

$$O_{K_{n,j,\tau}^m}^{all}(t) = \sum_{K_{n',j',\tau'}^m(t)=0}^{K_{n,j,\tau}^m(t)-1} o_{K_{n',j',\tau'}^m(t)} + o_{-2}^{m,re}(t) \cdot \{A, \theta_{n,j,\tau}^m(t) = 1\} \vee \{B\} + o_{-1}^{m,re}(t) \cdot \{B, \theta_{n,j,\tau}^m(t) = 1\}, \quad (11)$$

where $o_{-2}^{m,re}(t)$ and $o_{-1}^{m,re}(t)$ respectively represent the computing volume of the uncompleted part of the task in the PZ and the SZ at the beginning of time slot t on CPN node m . $\{\cdot\}$ is an indicator function, only the conditions within it are satisfied, $\{\cdot\} = 1$, otherwise $\{\cdot\} = 0$. $\{A, \theta_{n,j,\tau}^m(t) = 1\}$ means the task $K_{n,j,\tau}^m(t)$ is marked as a preemptive task under the *Situation A*. $\{B\}$ means the CPN node m is in the *Situation B*. Especially, when task -2 exists in the PZ, if the SZ is empty, task -2 has to wait for the preemptive tasks to be computed first, so we have:

$$O_{-2}^{all} = \{A\} \cdot \sum_{K_{n',j',\tau'}^m(t)=0}^{\Phi_m(t)} o_{K_{n',j',\tau'}^m(t)}. \quad (12)$$

If task -1 exists in SZ, it has to wait for the task -2 and the preemptive tasks (if any) to be executed first, so we have:

$$O_{-1}^{all} = o_{-2}^{m,rec} + \sum_{K_{n',j',\tau'}^m(t)=0}^{\Phi_m(t)} o_{K_{n',j',\tau'}^m(t)}. \quad (13)$$

Then, the related delay can be defined as follows.

Wireless Transmission Delay: The delay task $K_{n,j,\tau}^m(t)$ is offloaded to CPN node m from the device by the wireless channel, which can be determined as:

$$D_{K_{n,j,\tau}^m}^{wtran}(t) = D_{\varepsilon_{n,j}(\tau)}^{wtran}(t), \quad (14)$$

Transmission Delay: The delay task $K_{n,j,\tau}^m(t)$ transmit to CPN node m from n , which can be determined as:

$$D_{K_{n,j,\tau}^m}^{tran}(t) = \sum_{l \in P_{n,m}(\tau)} \frac{d_{n,j,\tau}}{B_l(\tau)}, \quad (15)$$

Computing Delay: The delay for task $K_{n,j,\tau}^m(t)$ to complete computing at the destination CPN node m , which can be defined as:

$$D_{K_{n,j,\tau}^m}^{comp}(t) = D_{\varepsilon_{n,j}(\tau)}^{comp}(t), \quad (16)$$

Slot Queuing Delay: The queuing time in the CQ of task $K_{n,j,\tau}^m(t)$ in time slot t (from the beginning to the end during the time slot t), which can be calculated by:

$$D_{K_{n,j,\tau}^m}^{slotqueue}(t) = \begin{cases} 0, & K_{n,j,\tau}^m(t) < 0. \\ \min\left(\frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}, \Delta T\right), & K_{n,j,\tau}^m(t) > 0. \end{cases} \quad (17)$$

The tasks in the PZ or SZ will not experience queuing in the CQ, thus their slot queuing delay are 0; For task $K_{n,j,\tau}^m(t)$ in the CQ, if the tasks computed before it can be completed within ΔT , then the task $K_{n,j,\tau}^m(t)$ can enter the PZ for computing at

time slot t , so the $D_{K_{n,j,\tau}^m}^{slotqueue}$ is $\frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}$. Otherwise, the task $K_{n,j,\tau}^m(t)$ cannot enter the PZ before the end of time slot t , thus the $D_{K_{n,j,\tau}^m}^{slotqueue}$ should be ΔT .

Slot Computing Delay: The computing delay of the task $K_{n,j,\tau}^m(t)$ within time slot t , which is defined as:

$$D_{K_{n,j,\tau}^m}^{slotcomp}(t) = \begin{cases} D_{K_{n,j,\tau}^m}^{comp}(t) - D_{K_{n,j,\tau}^m}^{acomp}(t-1), & t^* = t \\ \max\left(\Delta T - \frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}, 0\right), & t^* > t \end{cases}, \quad (18)$$

where $D_{K_{n,j,\tau}^m}^{acomp}(t-1)$ is the accumulated computing delay of task $K_{n,j,\tau}^m(t)$ before the end of time slot $t-1$, t^* is specifically refers to the completing time slot of task $K_{n,j,\tau}^m(t)$. $t^* = t$ means the task can be completed in time slot t , and $t^* > t$ indicates that the task cannot be completed in time slot t . For the task with $t^* = t$, its slot computing delay should be the computing delay minus the accumulated computing delay before the end of time slot $t-1$. For the task with $t^* > t$, if the tasks computed before it can be completed during time slot t , it will enter PZ and the $D_{K_{n,j,\tau}^m}^{slotcomp}(t)$ is $\Delta T - \frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}$, otherwise, the $D_{K_{n,j,\tau}^m}^{slotcomp}(t)$ is 0.

Slot Suspension Delay: The suspension delay of task $K_{n,j,\tau}^m(t)$ in time slot t , which can be denoted by:

$$D_{K_{n,j,\tau}^m}^{slotsusp}(t) = \begin{cases} 0, & K_{n,j,\tau}^m(t) > 0. \\ \min\left(\frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}, \Delta T\right), & K_{n,j,\tau}^m(t) < 0. \end{cases} \quad (19)$$

The tasks that go through the suspension phase can only be the tasks that initially exist in the SZ or PZ tasks at time slot t . Therefore, the slot suspension delay of tasks in CQ are 0; for the task initially in PZ or SZ at time slot t , if the tasks (such as those preemptive tasks) computed before it can be completed within the duration ΔT , the task can leave the SZ and enter PZ , so the time slot suspension delay is $\frac{O_{K_{n,j,\tau}^m}^{all}(t)}{f_m}$, otherwise, the task cannot leave the SZ in the current time slot, and its time slot suspension delay is ΔT .

Accumulated Queuing Delay: The cumulative queuing delay of task $K_{n,j,\tau}^m(t)$ in the CQ before the end of time slot t . It can be calculated as follows.

$$D_{K_{n,j,\tau}^m}^{queue}(t) = D_{K_{n,j,\tau}^m}^{queue}(t-1) + D_{K_{n,j,\tau}^m}^{slotqueue}(t). \quad (20)$$

Accumulated computing delay: The cumulative computing time of task $K_{n,j,\tau}^m(t)$ before the end of time slot t . It can be calculated as follows.

$$D_{K_{n,j,\tau}^m}^{acomp}(t) = D_{K_{n,j,\tau}^m}^{acomp}(t-1) + D_{K_{n,j,\tau}^m}^{slotcomp}(t). \quad (21)$$

Accumulated Suspension Delay: The cumulative suspension delay of task $K_{n,j,\tau}^m(t)$ before the end of time slot t . It can be calculated as follows.

$$D_{K_{n,j,\tau}^m}^{susp}(t) = D_{K_{n,j,\tau}^m}^{susp}(t-1) + D_{K_{n,j,\tau}^m}^{slotsusp}(t). \quad (22)$$

Accumulated Waiting Delay: The delay that tasks are scheduled to the destination nodes but outside the PZ before the end of time slot t , which can be calculated as follows.

$$D_{K_{n,j,\tau}^m}^{wait}(t) = D_{K_{n,j,\tau}^m}^{queue}(t) + D_{K_{n,j,\tau}^m}^{susp}(t). \quad (23)$$

Due to the fact that a task may span multiple time slots, the complete waiting delay can only be given at its completion time slot t^* .

Processing Delay: The total delay taken from the birth of the task $K_{n,j,\tau}^m(t^*)$ to its completion, which is denoted as:

$$D_{K_{n,j,\tau}^m}^{pro}(t^*) = D_{K_{n,j,\tau}^m}^{wtran}(t^*) + D_{K_{n,j,\tau}^m}^{tran}(t^*) + D_{K_{n,j,\tau}^m}^{wait}(t^*) + D_{K_{n,j,\tau}^m}^{comp}(t^*). \quad (24)$$

Remaining Computing Delay: At the beginning of time slot t , the delay required for task $K_{n,j,\tau}^m(t)$ to complete the unfinished portion, which can be calculated as:

$$D_{K_{n,j,\tau}^m}^{remcomp}(t) = D_{K_{n,j,\tau}^m}^{comp}(t) - D_{K_{n,j,\tau}^m}^{acomp}(t-1). \quad (25)$$

Left Delay: At the beginning of time slot t , the left delay of the task, which is expressed as:

$$D_{K_{n,j,\tau}^m}^{left}(t) = D_{K_{n,j,\tau}^m}^{tllr}(t) - (t - \tau). \quad (26)$$

V. PROBLEM FORMULATION AND ALGORITHM DESIGN

In this section, we first determine our optimization objective. Then, we describe the scheduling problem with DRL. Lastly, we design a DRL algorithm to solve our scheduling problem.

A. Optimization Objective

1) **Success Rate:** For users, the primary objective is the successful processing of computing tasks. Efficient task allocation, which maximizes the number of tasks processed successfully, is a key indicator of the system's performance. Consequently, we define the task success rate as one of the optimization objectives. A task is considered successful if it is completed within the specified tolerable delay; otherwise, it is classified as a failed task. Thus, the success rate can be expressed as:

$$\mathbb{E}(t) = \frac{H^{suc}(t)}{H^{suc}(t) + H^{fail}(t)}, \quad (27)$$

where $H^{suc}(t)$ and $H^{fail}(t)$ respectively denote the number of successful and failed tasks in time slot t .

2) **Average Processing Delay:** Delay is a critical factor influencing user experience, and minimizing task processing delay is essential for improving overall system performance. Thus, the average processing delay of tasks is defined as one of our Optimization objectives, which can be expressed as:

$$\mathbb{D}(t) = \frac{1}{H_m^{suc}(t)} \sum_{m=1}^N \sum_{K_{n,j,\tau}^m(t)=-2}^{K_{n,j,\tau}^m(t)=t} \left(D_{K_{n,j,\tau}^m}^{pro}(t) \cdot \{t^* = t\} \right), \quad (28)$$

where $t^* = t$ means the task $K_{n,j,\tau}^m(t)$ can be completed execution in time slot t . $K_{n,j,\tau}^m(t)$ is the max number of tasks in the CQ of the CPN node m at time slot t .

3) *Load Balancing*: From the user's perspective, the preference is to allocate tasks to CPN nodes with higher computing power. However, concentrating all computing tasks on these nodes may lead to excessive load, risking node failure and partial network congestion. Meanwhile, other nodes may remain under-utilized, resulting in an imbalance of computing resources. Such a situation is undesirable from the perspective of the computing power provider, who aims to optimize resource utilization across the CPNs. Therefore, considering the difference in computing power, the load balancing factor is defined as the disparity in resource occupancy, which is given by:

$$\mathbb{W}(t) = \sqrt{\frac{\sum_{m=1}^N (W_m(t) - \bar{W}(t))^2}{N}}, \quad (29)$$

where

$$W_m(t) = \frac{1}{\Delta T \cdot f_m} \left(o_{-2}^{m,rec} + o_{-1}^{m,rec} + \sum_{K_{n,j,\tau}^m(t)=1}^{K_{cache}^{m,t}} o_{K_{n,j,\tau}^m(t)}^{m,rec} \right), \quad (30)$$

and

$$\bar{W}(t) = \frac{\sum_{m=1}^N W_m(t)}{N} \quad (31)$$

B. DRL for Task Scheduling

Based on the aforementioned optimization objectives, we are committed to identifying the optimal scheduling decisions that ensure a high task success rate within CPN systems, minimize the average processing delay of global tasks, and maintain load balancing across CPN nodes. Thus, the scheduling problem can be expressed as:

$$\mathbf{P1}: \max\{\mu_1 \mathbb{E}(t) - \mu_2 \mathbb{D}(t) - \mu_3 \mathbb{W}(t)\}, \quad (32)$$

where, μ_1 , μ_2 and μ_3 are the weight factors.

However, the tasks within the CPN system exhibit considerable heterogeneity and are characterized by high concurrency. Additionally, the CPN nodes are heterogeneous, and the network environment is time-varying, which further complicates task management. Moreover, the incorporation of P2M at the execution level introduces additional complexities. Specifically, the problem is marked by a high-dimensional state space and an extensive solution space, rendering **P1** an NP-hard problem. As a result, finding the optimal solution for **P1** presents a significant challenge.

Fortunately, DRL, known for its exceptional interaction and decision-making capabilities, has demonstrated notable performance in addressing previous scheduling challenges [10], [12], [13], providing a promising approach to this problem. DRL combines the perceptual power of deep learning with the decision-making capabilities of reinforcement learning. By continuously interacting with the environment, the DRL agent learns from experience and leverages its strong ability in learning and fitting value functions to provide real-time optimal decisions based on past interactions. This makes DRL particularly well-suited for solving scheduling problems in high-dimensional state

and solution spaces. Therefore, we introduce DRL to model and solve the scheduling problem.

In DRL, it is crucial to define the system's state, actions, and rewards, which are described as follows.

State: To ensure the CPN controller has a comprehensive perception of the system, it is necessary to fully capture the network status, the newly arrived tasks, and the current state of the CPN nodes. This global view of the system enables the controller to make informed decisions. Therefore, the state at time slot t can be defined as follows.

$$\mathcal{S}(t) = \{\{B_l(t), l \in \mathcal{L}\}, \{Q_n^{arr}(t), Q_n^{cache}(t), Q_n^{pro}(t), Q_n^{sup}(t) \mid n \in \mathcal{N}\}\}, \quad (33)$$

where $Q_n^{arr}(t)$, $Q_n^{cache}(t)$, $Q_n^{pro}(t)$ and $Q_n^{sup}(t)$ respectively represent the related information about the tasks in the AQ, CQ, PZ and SZ of the CPN node n at the beginning of time slot t .

Action: Action refers to the scheduling decision formulated by the CPN controller, which determines where the newly arrived tasks in the AQ will ultimately be executed. The action at time slot t can be represented as:

$$\mathcal{A}(t) = \{a_{n,j}^m(t) \in \{0, 1\} \mid n, m \in \mathcal{N}, 1 \leq j \leq J\}. \quad (34)$$

Reward: In DRL, the agent requires feedback from the environment after each interaction to assess the impact of its scheduling decisions on the system, thereby enabling continuous improvement. To this end, based on the optimization objectives and problem **P1**, we define the instant reward that the scheduling policy provides to the system at time slot t as:

$$\mathcal{R}(t) = \mu_1 \mathbb{E}(t) - \mu_2 \mathbb{D}(t) - \mu_3 \mathbb{W}(t), \quad (35)$$

Q-function: This function represents the action-value, evaluating the long-term effects of employing strategy π for formulating action in a given state, which is given as follows.

$$Q_t^\pi[\mathcal{S}(t), \mathcal{A}(t)] = \lim_{T \rightarrow \infty} \frac{1}{T} E_\pi \left[\sum_{l=0}^T \gamma^l R(t+l) \mid \mathcal{S}(t), \mathcal{A}(t) \right], \quad (36)$$

where $\gamma \in [0, 1]$ is a discount factor.

The CPN system entails long-term task scheduling across multiple time slots, constituting a continuous process. Therefore, to ensure the long-term stability and efficiency of the policy-formulating within the CPN system, we aim to identify a stable strategy π^* that consistently improves system performance and maximizes the long-term reward. Thus, the problem **P1** can be transformed and expressed as:

$$\mathbf{P2}: \pi^* = \arg \max Q_t^\pi(\mathcal{S}(t), \mathcal{A}(t)). \quad (37)$$

To solve the **P2**, we adopt the TD3 among numerous DRL algorithms to formulate an optimal strategy π^* through extensive training. As an enhancement over the deep deterministic policy gradient (DDPG), TD3 addresses the overestimation issue [38], [39] and improves stability through the following key improvements:

1) *Twin Critic Networks*: TD3 introduces two independent critic networks to estimate the Q-value (i.e., action value). Specifically, it selects the smaller Q-value between the two

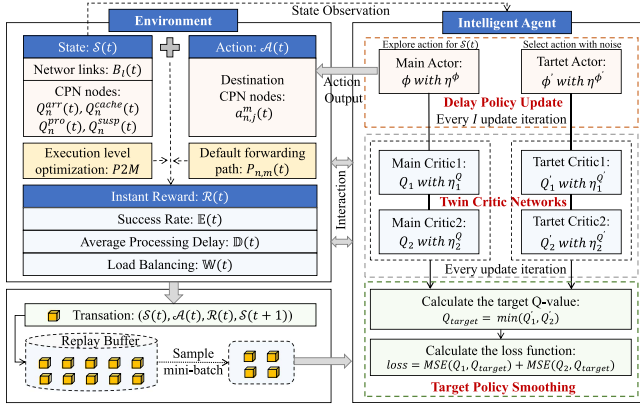


Fig. 3. Structure of P2TD3 algorithm.

critics to determine the target Q-value, effectively mitigating the issue of overestimation.

2) *Delayed Policy Updates*: TD3 abandons the alternating update strategy of actor and critic networks in DDPG and adopts a delayed update strategy. Specifically, the update frequency of the critic network is higher than the actor network, meaning that the actor network only updates once after multiple updates of the critic network. This asynchronous update aims to make the critic more fitted to the value function before guiding the actor's policy-formulating on actions, thereby reducing policy fluctuations and errors caused by frequent updates.

3) *Target Policy Smoothing*: TD3 introduces noise into the target strategy, which is equivalent to adding regularization terms to the critic network. Specifically, when calculating the target value, adding perturbations to the actions of the next state helps to avoid being influenced by certain extreme value functions and enhances the stability of the critic network learning.

C. Design of P2TD3 Algorithm

The improvements mentioned above make TD3 more suitable for formulating long-term stable and optimal strategies. To adapt to our CPN system, we further design the P2TD3 algorithm (the structure is shown in Fig. 3) specifically tailored for the scheduling problem under P2TS, which is detailed in Algorithm 2. The key steps are as follows.

1) *Establish Two Sets of Networks*: P2TD3 establishes and maintains two sets of networks. The first set, referred to as the main networks, is responsible for interacting with the environment, outputting actions based on the current state, and evaluating the Q-value corresponding to given states and actions. This set consists of an actor network ϕ and two critic networks Q_1, Q_2 with respective parameters η^ϕ, η_1^Q and η_2^Q . The second set is the target network, which provides a stable target Q-value during the training process and is used to calculate the loss of the main network. This set contains an actor network ϕ' and two critic networks Q'_1 and Q'_2 with parameters $\eta^{\phi'}, \eta_1^{Q'}$ and $\eta_2^{Q'}$.

2) *State Observation*: P2TD3 requires observing environmental state information as input for interacting with the environment. At the beginning of time slot t , new tasks access the

Algorithm 2: The Proposed P2TD3 Algorithm.

Input: The scheduling state and algorithm parameters.

Output: The scheduling policies.

```

1 Initialize the main networks and target networks  $\phi, Q_1, Q_2, \phi', Q'_1$  and  $Q'_2$  with parameters  $\eta^\phi, \eta_1^Q, \eta_2^Q, \eta^{\phi'}, \eta_1^{Q'}, \eta_2^{Q'}$ ;
2 for each episode  $ep = 1, 2, \dots, E$  do
3   Reset the tasks and normalize the initial state  $s_0$ ;
4   for  $t = 1, 2, \dots, T$  do
5     New computing tasks access the CPN system;
6     Observe the current environment state  $s_t$  with tuple  $\{B_l(t), Q_m^{arr}(t), Q_m^{cache}(t), Q_m^{pro}(t), Q_m^{usp}(t)\}$ ;
7     Calculate the shortest path between CPN nodes via the Floyd algorithm;
8     Generate and  $a_t$  with  $\{a_{n,j}^m(t)\}$  via Eq. (38);
9     Forward related tasks to the destination CPN nodes according to  $a_t$  through the shortest path;
10    Define priorities of tasks in the CPN system via Eq. (7);
11    Reorder the tasks in the CQ according to priorities;
12    Implement preemptive mechanism via Algorithm 1;
13    Compute the instant reward  $r_t$ ;
14    Observe and normalize the next state  $s_{t+1}$ ;
15    if the replay buffer does not overflow then
16      Store the transition  $(s_t, a_t, r_t, s_{t+1})$  in RB;
17    else
18      Replace a transition in RB randomly;
19      Sample a mini-batch of transitions with  $(s_t, a_t, r_t, s_{t+1})$  from RB randomly;
20      Select a scheduling action with  $\mathbb{N}$  via Eq. (39);
21      Compute the target  $Q$  value via Eq. (40);
22      Update the main critic networks via Eq. (41);
23    if  $t \bmod I$  then
24      Update the actor network via Eq. (42);
25      Soft-update the target network via Eq. (43);

```

CPN systems, and the current state s_t is observed. s_t reflects the current environment information, including the $B_l(t), Q_n^{arr}(t), Q_n^{cache}(t), Q_n^{pro}(t)$ and $Q_n^{usp}(t)$.

3) *Policy Exploration*: Once receive s_t , the main actor network explores an action a_t :

$$a_t \sim \pi_\phi(s_t) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2). \quad (38)$$

where, $\pi_\phi(s_t)$ is the action selection strategy with ϕ under s_t , $\mathcal{N}(0, \sigma^2)$ is a Gaussian distribution with a mean of 0 and a variance of σ^2 .

4) *Execution Level Optimization*: After the scheduling policy is issued to the environment, the tasks in the *AQs* of CPN nodes are scheduled to the destination CPN nodes and enter the *CQ*. Then, the P2M takes effect to manage the execution of the tasks at time slot t .

5) *Experience Storage*: Once the execution of tasks is determined, the instant reward r_t can be calculated to quantify and evaluate the benefits brought to the system by a_t under s_t . Then, the transition with a tuple (s_t, a_t, r_t, s_{t+1}) is stored to the experience replay buffer *RB*.

6) *Networks Updating*: Sample a mini-batch size of B from the experience replay buffer and select an action by:

$$\tilde{a} = \pi_\phi(s_{t+1}) + \epsilon', \epsilon' \sim \text{clip}(\mathbb{N}(0, \sigma'^2), -C, C), \quad (39)$$

where the action $\tilde{a}$ is clipped within the range of $(-C, C)$. Then we calculate the target Q -value by:

$$y = r_t + \gamma \min_{i=1,2} \{Q'_i(s_{t+1}, \tilde{a})\}. \quad (40)$$

The two critical network parameters will be updated by:

$$\eta_i^{Q'} \leftarrow \arg \min_{\eta_i} \frac{1}{B} \sum (y - Q_i(s_t, a_t))^2. \quad (41)$$

The actor network is trained via the policy gradient by:

$$\nabla_\phi \mathbb{J}(\phi) = \frac{1}{B} \sum \nabla_{a_t} Q_1(s_t, a_t) \Big|_{a_t=\pi_\phi(s_t)} \nabla_\phi \pi_\phi(s_t). \quad (42)$$

where, $\mathbb{J}(\phi)$ is the loss function of ϕ . Lastly, update the target network every I iteration:

$$\eta_i^{Q'} \leftarrow \beta \eta_i^{Q'} + (1 - \beta) \eta_i^{Q'}, \beta \in (0, 1), i = 1, 2. \quad (43)$$

VI. PERFORMANCE EVALUATION

In this section, we first design our simulation environment. Then we introduce four baselines that compare with the proposed P2TD3 algorithm. Eventually, we provide a series of simulation results to evaluate the performance of our strategy.

A. Simulation Setup

We consider a CPN system that consists of 20 CPN nodes and 30 CPN routers. The interconnection between them forms the network links of the CPNs system. Given the diverse computing delay requirements of heterogeneous tasks, we set the duration of a single scheduling period to 2 seconds. To thoroughly assess algorithms' performance across continuous scheduling periods, we configure the simulation to run for 30 consecutive periods. Partly referring to works [13], [33], [35], [40], [41], the additional parameter settings are detailed in Table I. Based on these settings, we construct the simulation environment using Python and the PyTorch framework. Then, we introduce four baseline algorithms for simulation comparison, which are as follows.

- P2DDPG: Utilizes DDPG for scheduling, incorporating the P2M.
- SPTD3: Utilizes TD3 for scheduling, incorporating a single indicator (tolerable delay) priority planning.

TABLE I
LIST OF NOTATIONS

Notation	Description
$\mathcal{N}, N, n, m$	Set, number and index of CPN nodes
f_n	Computing power of CPN node n
$\mathcal{R}, R, r$	Set, number and index of routers
$\mathcal{L}, L, l$	Set, number and index of network links
$B_l(t)$	Bandwidth of link l
$\varepsilon_n(t), \varepsilon_{n,j}(t)$	Set of tasks, j -th task newly accessing CPN node n in slot t
$d_{n,j,t}, o_{n,j,t}$	Data and computing volume of task $\varepsilon_{n,j}(t)$
t, τ, t^*	Index of current, task generated, and task completed slot
$p_{n,j,\tau}(t)$	Priority of the task $\varepsilon_{n,j}(\tau)$ in slot t
$K_{n,j,\tau}^m(t)$	Unique sequence number of task $\varepsilon_{n,j}(\tau)$ after priority planning
$\theta_{n,j,\tau}^m(t)$	Flag for whether task $K_{n,j,\tau}^m(t)$ becomes a preemptive task
Υ_a, Υ_b	Delay comparison function situation A, B
$O_{K_{n,j,\tau}^m}^{all}$	Total computing volume of tasks should be computed before task $K_{n,j,\tau}^m(t)$
$o_{-2}^{m,re}(t), o_{-1}^{m,re}(t)$	Remaining computing volume of task in the PZ and SZ on CPN node m
$o_{K_{n,j,\tau}^m}^{m,t}$	Computing volume of task $K_{n,j,\tau}^m(t)$
$\Phi_m(t), K_{cache}^{m,t}$	Maximum sequence of preemptive tasks and tasks in the CQ at slot t
$P_{n,m}(t)$	Default forwarding path from CPN node n to m at slot t
$D_{K_{n,j,\tau}^m}^{remcomp}(t)$	Remaining computing delay of task $K_{n,j,\tau}^m(t)$
$D_{K_{n,j,\tau}^m}^{left}(t)$	Left delay of task $K_{n,j,\tau}^m(t)$
$D_{K_{n,j,\tau}^m}^{tolr}(t)$	Tolerable delay of task $K_{n,j,\tau}^m(t)$

- FCFSTD3: Utilizes TD3 for scheduling, incorporating the FCFS basis for task execution.
- P2R: Employs a random distribution for scheduling, incorporating the P2M.

B. Simulation Results and Analysis

1) *Priority Factors Performance*: First, we evaluate the effectiveness and necessity of the introduced priority factors, examining whether they promote the prioritized execution of computing tasks indeed in specific situations. In each time slot, we assume that computing tasks are randomly scheduled to their destination CPN nodes and enter the *CQ*. These tasks follow the proposed multi-factor dynamic priority planning, where priorities are defined by (7) and tasks are queued in descending order of priority for computing. During this process, preemption between tasks is not supported. To investigate the individual impact of each priority factor, we assign the priority weight factors $\omega_1, \omega_2, \omega_3$, and ω_4 to four different configurations:

- E: $\omega_1 = 1, \omega_2 = 0, \omega_3 = 0, \omega_4 = 0$;
- S: $\omega_2 = 1, \omega_1 = 0, \omega_3 = 0, \omega_4 = 0$;
- G: $\omega_3 = 1, \omega_1 = 0, \omega_2 = 0, \omega_4 = 0$;
- U: $\omega_4 = 1, \omega_1 = 0, \omega_2 = 0, \omega_3 = 0$.

These configurations ensure that each task's computing is influenced by only one priority factor.

The priority factors are introduced to ensure that tasks with higher priority are executed first, improving their success rate. In

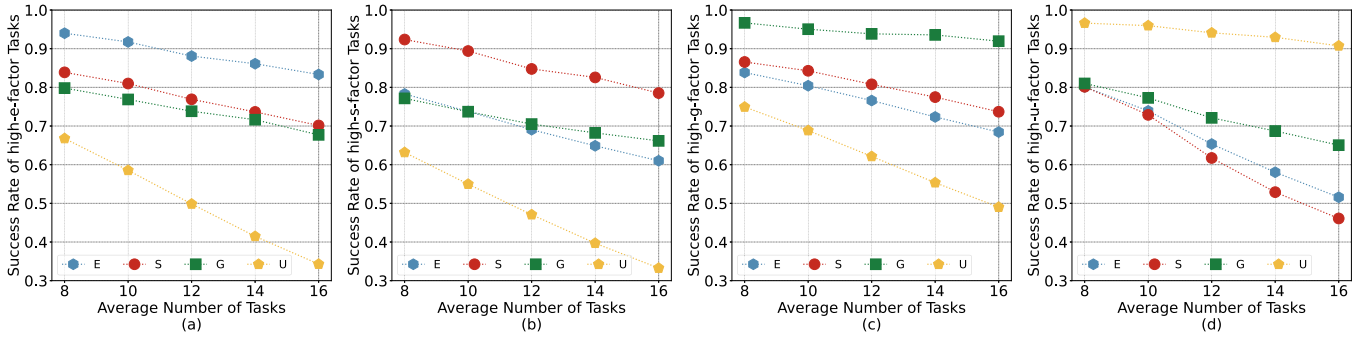


Fig. 4. Illustration of the priority factors performance.

the simulation, we focus on the top 20% of tasks with the highest values for task type, delay elasticity, relative execution speed, and task lifespan ratio, referred to as high- e -factor, high- s -factor, high- g -factor, and high- u -factor tasks, respectively. After 500 simulation episodes, the average success rate, along with the variation in the average number of tasks accessing CPN nodes at the beginning of each time slot, is presented in Fig. 4.

Fig. 4(a) shows that high- e -factor tasks achieve the highest success rate under configuration E, as task priority is determined solely by task type, allowing tasks with higher e values to enjoy computing resources first.

In Fig. 4(b), the high- s -factor tasks, which have lower tolerable delay, show a significantly higher success rate than those in the other configurations. This can be attributed to the delay elasticity factor, which directionally increases the priority of tasks with lower tolerable delay, allowing them to be computed first. We notice that other factors may influence the success rate of tasks with lower tolerable delay to some degree, but this is mainly due to the correlation of some static attributes within the task at the time of generation, which is highly random and cannot achieve targeted priority improvement. For example, when task priority is determined solely by task type, those tasks with higher task type values and lower tolerable delay will show a higher success rate. However, not all tasks with lower tolerable delay have high task type values, and in such cases, the success rate will remain lower.

In Fig. 4(c), when priority is determined by relative execution speed, high- g -factor tasks, with smaller computing volume, show the highest success rate.

Fig. 4(d) reveals that high- u -factor tasks, which have been waiting longer, perform better under configuration U. This indicates that the task lifespan ratio effectively enhances the priority of long-waiting tasks, preventing failure due to excessive waiting.

These simulations confirm that the four priority factors we introduced are both effective and necessary. Each factor directionally enhances task priority in specific aspects, and their weighted combination provides flexible priority planning for different scenarios.

As mentioned earlier, our objective is to ensure high task success rates and minimize average processing delay in CPNs. This requires completing as many tasks as possible within their tolerable delay, while reducing processing delay to improve

overall processing efficiency. In addition, we also take fairness into account to some extent. Therefore, we set the weight factors as follows for subsequent simulations: $\omega_1 = 0.2$, $\omega_2 = 0.4$, $\omega_3 = 0.3$, $\omega_4 = 0.1$. Notably, ω_4 is set to the lowest value to prevent tasks with long waiting time from increasing priority too quickly. Otherwise, it may result in a large number of long-waiting tasks being computed first, causing significant failures in more urgent tasks.

2) *Convergence Performance*: Before the formal deployment of DRL algorithms, a series of offline training sessions are required to iteratively optimize and refine the decision models. Therefore, we conducted offline training with 500 episodes for different algorithms under five different random seeds. Each episode contains 30 consecutive task scheduling periods, and at the beginning of each period, the average number of tasks newly accessing each CPN node is set as 8. As shown in Fig. 5, solid lines represent the average cumulative reward of the under five random seeds and shaded areas represent the fluctuation range.

In Fig. 5(a), we evaluate the impact of different learning rates (LRs) on the P2TD3 algorithm. A higher LR leads to faster convergence but increases fluctuation and instability, while a lower LR results in slower convergence. We observe that an LR of $5.00e-6$ offers moderate convergence speed, high rewards, and stability. Therefore, we use $5.00e-6$ as the default LR for subsequent training with other algorithms.

Fig. 5(b) shows the performance of the scheduling algorithms based on TD3 with the same LR. The convergence speeds of the three algorithms are similar, but P2TD3 shows the highest return and the smallest fluctuation.

Fig. 5(c) compares scheduling algorithms based on TD3 and DDPG. P2DDPG is demonstrated to be inferior in terms of return and stability due to its own limitations.

3) *Related Indicators With Task Number*: After training, we deploy the algorithms and perform scheduling tests for 50 episodes when the average number of tasks accessing CPN nodes varies between $\{8, 10, 12, 14, 16\}$. Then, we draw the curves in Fig. 6. The increase of task number puts higher demands on computing power and expands the solution space, presenting challenges to scheduling decisions. This results in higher average task processing delays, a gradual decline in task success rates, and poorer node load balancing, ultimately decreasing the cumulative reward.

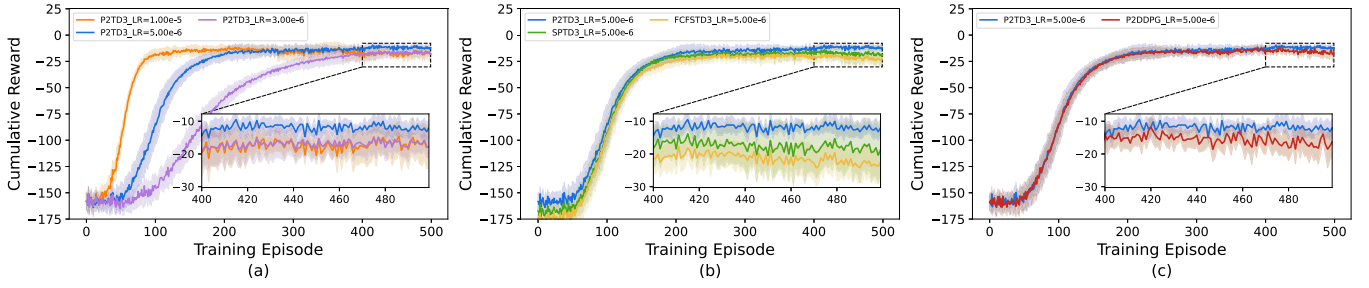


Fig. 5. Illustration of the convergence performance.

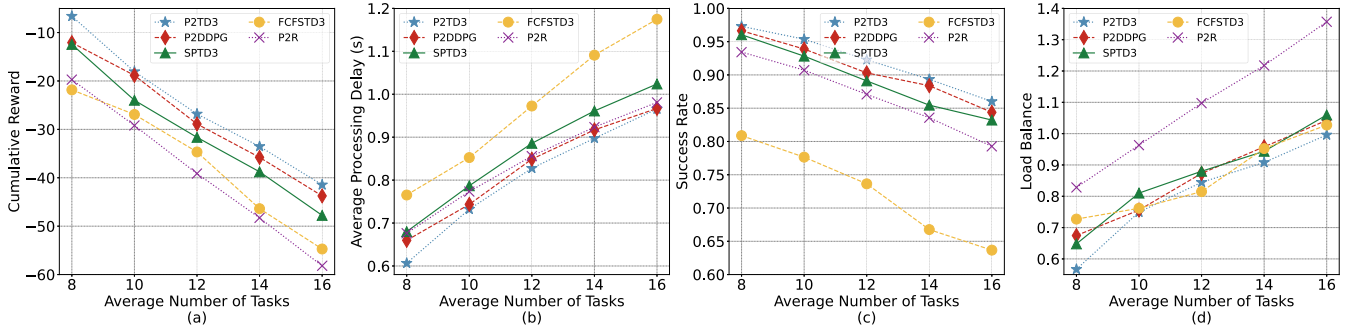


Fig. 6. Illustration of optimization indicators with different algorithms under different numbers of tasks.

Compared to the baseline algorithm, the proposed P2TD3 algorithm achieves the highest cumulative reward (Fig. 6(a)) and task success rate (Fig. 6(c)), as well as the lowest average task processing delay (Fig. 6(b)). Moreover, it demonstrates exceptional performance in load balancing (Fig. 6(d)). Its excellent performance can be attributed to the dual optimization at the task execution and scheduling levels. At the execution level, its support for P2M ensures high-priority tasks can preempt computing resources for priority execution, overcoming “forced waiting” and avoiding large-scale timeouts of urgent tasks, which brings ultra-low delay and ultra-high success rate. At the scheduling level, as an enhanced version of DDPG, the TD3-based algorithm addresses the limitations of DDPG and offers a more stable and effective exploration of optimal scheduling decisions. P2DDPG is slightly worse than P2TD3, especially in terms of success rate and load management (Fig. 6(d)). The lack of P2M in SPTD3 hinders its ability to avoid “forced waiting,” leading to potential timeouts for urgent tasks and relatively higher average task processing delays. Furthermore, the FCFS execution manner of FCFSTD3 is weak in coping with tasks that arrive in a highly random manner, resulting in the worst performance in terms of average processing delay (Fig. 6(b)) and success rate (Fig. 6(c)). As the default execution method for post-scheduling tasks in many studies, the simulation results highlight its limitations. Specifically, it lacks the flexibility to accommodate the heterogeneity of computing tasks and fails to provide adaptive execution strategies tailored to diverse task requirements. P2R with random scheduling performs the worst in cumulative rewards (Fig. 6(a)) and load balancing (Fig. 6(d)).

To further analyze the performance of the algorithms, we explore additional key metrics as the average number of tasks varies, as shown in Fig. 7. In Fig. 7(a), it is evident that the P2TD3 algorithm results in the lowest average queuing delay, minimizing delay waste. Priority planning within P2M takes the computing volume of tasks as one of the factors for defining priority. This may allow tasks with lower computing volume to be executed first, which in turn releases computing resources earlier and reduces the queueing delay for subsequent tasks. Additionally, P2TD3’s ability to overcome “forced waiting” further reduces the average queuing delay. Similarly, P2R also performs well in this metric. In contrast, SPTD3, which uses single-factor priority planning, exhibits poor performance. Lacking execution-level optimization, FCFSTD3 introduces the highest average queuing delay. We define the value obtained by subtracting the actual processing delay from the tolerable delay as delay savings. P2TD3 performs best in this value (Fig. 7(b)), indicating that it minimizes delay waste and facilitates more real-time processing. Finally, to evaluate the algorithms’ impact on those tasks with relatively high priority, we focus on the tasks with the top 10% of priority in each time slot, referred to as high-priority tasks, and analyze their success rate. As shown in Fig. 7(c), under both P2TD3 and P2DDPG, the success rate of the top 10% priority tasks exceeds 91%. In comparison to SPTD3 and FCFSTD3, the success rate of high-priority tasks under P2TD3 has increased by 6.06% to 29.25%, and by 70.71% to 77.28%, respectively. This advantage is particularly pronounced as the number of tasks increases, highlighting the superiority of P2TD3 in handling high-priority tasks.

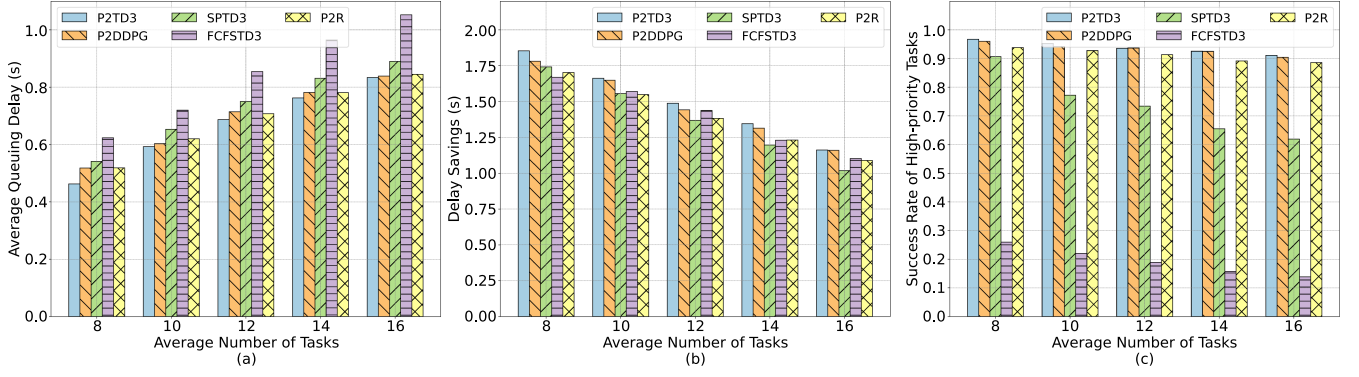


Fig. 7. Illustration of other key indicators with different algorithms under different numbers of tasks.

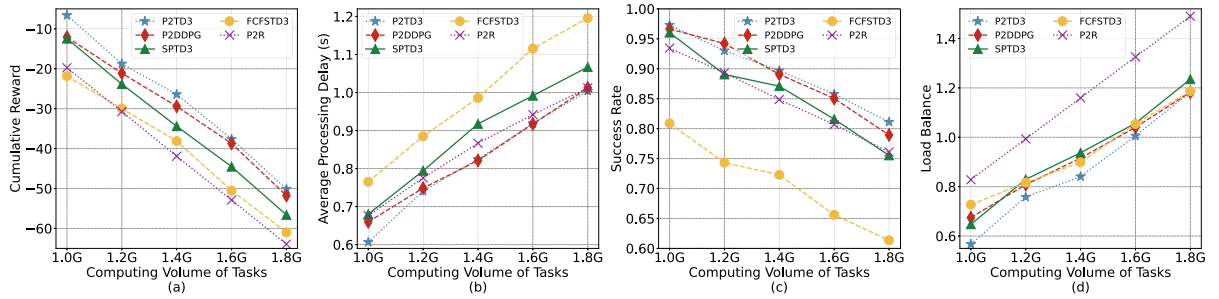


Fig. 8. Illustration of optimization indicators under different algorithms with the variation of computing volume per task.

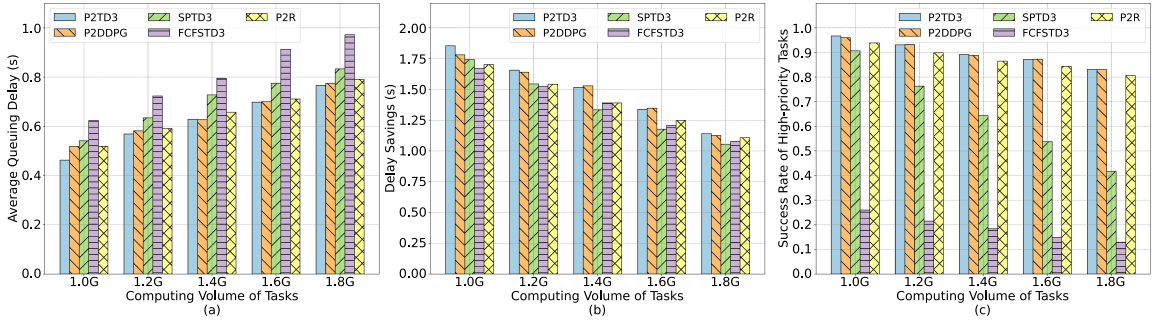


Fig. 9. Illustration of other key indicators under different algorithms with the variation of computing volume per task.

TABLE II
PARAMETERS AND VALUES

Parameter in P2TD3	Value	Parameter in the CPN	Value
Training episodes	500	Duration of each time slot (single period)	2s
Policy noise	1	Average number of tasks on each CPN node	{8,10,12,14,16}
Batch size	32	The computing power of CPN nodes	[4,10] GCPU/s
Actor learning rate	5.00e-6	Data volume per task	[0.5,3] MB
Critic learning rate	1.50e-5	Computing volume per task	[0.01,4.5] GCPU
Discount	0.95	Tolerable delay for delay sensitive tasks	[0.3,1.6] s
Noise clip	0.25	Tolerable delay for non delay sensitive tasks	[2.0,6.0] s
Explore noise	0.1	The bandwidth of links	[500,1000]Mbps

4) *Related Indicators With Task Computing Volume*: In addition to the number of tasks, we also investigate the impact of task computing volume on the related Indicators. The average number of tasks accessing CPN nodes in each time slot is fixed

at 8. Let G represent the base computing volume per task, which ranges from $[0.01, 4.5]G_{CPU}$ and is gradually scaled up to $1.8G$ in Fig. 8. Due to the limited computing resources, an increase in task computing volume results in a significant rise in the average processing delay of tasks (Fig. 8(b)) and a corresponding decrease in the task success rate (Fig. 8(c)). Furthermore, load balancing management becomes increasingly difficult (Fig. 8(d)).

The average queuing delay increases and the average delay savings slumps as the task computing volume rises (Fig. 9). Tasks with higher computing volume require more timely execution to prevent timeout. The proposed P2TD3 algorithm brings the lowest average queuing delay to the task and performs well in terms of delay savings, which undoubtedly reduces the risk of timeout and minimizes delay waste. SPTD3 and

FCFSTD3 perform poorly due to their own limitations. Higher and higher computing volume leads to the demand for real-time processing of high-priority tasks surge, which poses a greater challenge to ensuring the success rate of high-priority tasks. Benefiting from the optimization in execution and scheduling level, P2TD3 and P2DDPG are superior to other algorithms (Fig. 9(c)). Compared with SPTD3 and FCFSTD3, the success rate of high-priority tasks under P2TD3 has increased by 6.06% to 41.35%, and 70.71% to 72.26%, respectively. P2R maintains a relatively high success rate (Fig. 8(d)) due to the support from the P2M.

VII. CONCLUSION

In this work, we have explored the optimization opportunities of tasks in CPNs at the scheduling and execution levels and proposed a P2TS strategy for application in CPNs, which addresses the limitation in existing research that pays insufficient attention to task execution after scheduling. We first designed P2M for managing the execution of tasks after scheduling; Then, we introduced DRL to formulate the task scheduling problem in CPNs under P2M and specifically designed a P2TD3 algorithm for implementation. Finally, we conducted extensive simulations across multiple dimensions. The results from comprehensive numerical analyses demonstrate that the proposed P2TD3 algorithm surpasses baseline algorithms in convergence, stability, and overall performance, thereby confirming the effectiveness and validity of the P2TS strategy.

REFERENCES

- [1] X. Liu et al., "A systematic review of digital twin about physical entities, virtual models, twin data, and applications," *Adv. Eng. Informat.*, vol. 55, 2023, Art. no. 101876.
- [2] J. Ni, Y. Chen, Y. Chen, J. Zhu, D. Ali, and W. Cao, "A survey on theories and applications for self-driving cars based on deep learning methods," *Appl. Sci.*, vol. 10, no. 8, 2020, Art. no. 2749.
- [3] M. Hu, X. Luo, J. Chen, Y. C. Lee, Y. Zhou, and D. Wu, "Virtual reality: A survey of enabling technologies and its applications in IoT," *J. Netw. Comput. Appl.*, vol. 178, 2021, Art. no. 102970.
- [4] Y. Cao et al., "A comprehensive survey of AI-generated content (AIGC): A history of generative AI from GAN to ChatGPT," 2023, *arXiv:2303.04226*.
- [5] H. Xiao, C. Xu, Y. Ma, S. Yang, L. Zhong, and G.-M. Muntean, "Edge intelligence: A computational task offloading scheme for dependent IoT application," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 7222–7237, Sep. 2022.
- [6] J. Zhang, X. Zhou, T. Ge, X. Wang, and T. Hwang, "Joint task scheduling and containerizing for efficient edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 2086–2100, Aug. 2021.
- [7] S. Yukun et al., "Computing power network: A survey," *China Commun.*, vol. 21, no. 9, pp. 109–145, Sep. 2024.
- [8] Y. Liang, G. Xiao, Z. Hua, W. Xing, Y. He, and X. Cheng, "Survey of computing power network industry standards," in *Proc. China Automat. Congr.*, 2023, pp. 5583–5588.
- [9] M. Cheng, J. Li, and S. Nazarian, "DRL-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers," in *Proc. 23rd Asia South Pacific Des. Automat. Conf.*, 2018, pp. 129–134.
- [10] Z. Liu, X. Qiu, S. Zhang, S. Deng, and G. Liu, "Service scheduling based on edge computing for power distribution IoT," *Comput., Mater. Continua*, vol. 62, no. 3, pp. 1351–1364, 2020.
- [11] Q. Tang et al., "Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1063–1079, May 2024.
- [12] L. Feng, R. Xie, Q. Tang, and T. Huang, "Delay-prioritized task scheduling with load balancing in computing power networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2024, pp. 1–6.
- [13] Q. Tang et al., "SiaTS: A service intent-aware task scheduling framework for computing power networks," *IEEE Netw.*, vol. 38, no. 4, pp. 233–240, Jul. 2024.
- [14] G. Xie et al., "Joint task scheduling and intelligence optimization in CPN-enabled connected intelligence systems," in *Proc. IEEE Glob. Commun. Conf.*, 2023, pp. 1101–1106.
- [15] L. Cheng, Y. Wang, F. Cheng, C. Liu, Z. Zhao, and Y. Wang, "A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 422–432, May/Jun. 2024.
- [16] B. Lei, Z. Liu, X. Wang, M. Yang, and Y. Chen, "Computing network: A new multi-access edge computing," *Telecommun. Sci.*, vol. 35, no. 9, pp. 44–51, 2019.
- [17] ITU-T, "Computing power network-framework and architecture: Y. 2501," 2021. [Online]. Available: <https://www.itu.int/rec/T-REC-Y.2501-202109-1>
- [18] X. Tang et al., "Computing power network: The architecture of convergence of computing and networking towards 6G requirement," *China Commun.*, vol. 18, no. 2, pp. 175–185, 2021.
- [19] Y. Zhang, C. Cao, X. Tang, R. Pang, S. Wang, and X. Wen, "Programmable service system based on SiDaaS in computing power network," in *Proc. 5th Int. Conf. Hot Inf.-Centric Netw.*, 2022, pp. 67–71.
- [20] W. Sun, Z. Li, Q. Wang, and Y. Zhang, "FedTAR: Task and resource-aware federated learning for wireless computing power networks," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 4257–4270, Mar. 2023.
- [21] S. H. Sahraei, M. M. R. Kashani, J. Rezazadeh, and R. Farahbakhsh, "Efficient job scheduling in cloud computing based on genetic algorithm," *Int. J. Commun. Netw. Distrib. Syst.*, vol. 22, no. 4, pp. 447–467, 2019.
- [22] J. Yan et al., "Energy-aware systems for real-time job scheduling in cloud data centers: A deep reinforcement learning approach," *Comput. Elect. Eng.*, vol. 99, 2022, Art. no. 107688.
- [23] X. Chen et al., "A WOA-based optimization approach for task scheduling in cloud computing systems," *IEEE Syst. J.*, vol. 14, no. 3, pp. 3117–3128, Sep. 2020.
- [24] S. Meng et al., "Security-aware dynamic scheduling for real-time optimization in cloud-based industrial applications," *IEEE Trans. Ind. Informat.*, vol. 17, no. 6, pp. 4219–4228, Sep. 2020.
- [25] H. Yuan, J. Bi, W. Tan, M. Zhou, B. H. Li, and J. Li, "TTSA: An effective scheduling approach for delay bounded tasks in hybrid clouds," *IEEE Trans. Cybern.*, vol. 47, no. 11, pp. 3658–3668, Nov. 2016.
- [26] B. Gong and X. Jiang, "Deep reinforcement learning for dependent task offloading in mobile edge computing systems," in *Proc. IEEE Smartworld, Ubiquitous Intell. Comput., Scalable Comput. Commun., Digit. Twin, Privacy Comput., Metaverse, Auton. Trusted Veh.*, 2022, pp. 1576–1581.
- [27] T. Tao, J. Hou, and A. Nayak, "A GNN-DRL-based collaborative edge computing strategy for partial offloading," in *Proc. IEEE Glob. Commun. Conf.*, 2023, pp. 3717–3722.
- [28] J. Fang, Y. Chen, and S. Lu, "A scheduling strategy for reduced power consumption in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2020, pp. 1190–1195.
- [29] L. Xia, D. Guo, Y. Wang, D. Sun, W. Zhen, and C. Jing, "Optimal load scheduling based on mobile edge computing technology in 5G dense networking," in *Proc. 3rd Asia Conf. Comput. Commun.*, 2022, pp. 137–142.
- [30] M. Zhang, J. Cao, L. Yang, L. Zhang, Y. Sahni, and S. Jiang, "ENTS: An edge-native task scheduling system for collaborative edge computing," in *Proc. IEEE/ACM 7th Symp. Edge Comput.*, 2022, pp. 149–161.
- [31] M. S. Bali et al., "An efficient task allocation framework for scheduled data in edge based Internet of Things using hybrid optimization algorithm approach," *Phys. Commun.*, vol. 58, 2023, Art. no. 102047.
- [32] X. Ma, A. Zhou, S. Zhang, Q. Li, A. X. Liu, and S. Wang, "Dynamic task scheduling in cloud-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2116–2130, Apr. 2023.
- [33] R. Xie, H. Zhu, Q. Tang, Q. Chen, S. Qiao, and T. Huang, "Joint task scheduling and load balancing in computing power network-enabled edge computing systems," in *Proc. IEEE 8th Int. Conf. Comput. Commun.*, 2022, pp. 563–568.
- [34] Q. Chen, C. Yang, S. Lan, L. Zhu, and Y. Zhang, "Two-stage evolutionary search for efficient task offloading in edge computing power networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 30787–30799, Oct. 2024.
- [35] R. Xie et al., "Delay-prioritized and reliable task scheduling with long-term load balancing in computing power networks," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3359–3372, Nov./Dec. 2024.

- [36] K. Ding, J. Zhu, and W. Fan, "Dynamic and preemptive task offloading in edge-cloud computing systems," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2022, pp. 498–503.
- [37] M. S. Bali, K. Gupta, D. Gupta, G. Srivastava, S. Juneja, and A. Nauman, "An effective technique to schedule priority aware tasks to offload data on edge and cloud servers," *Meas.: Sensors*, vol. 26, 2023, Art. no. 100670.
- [38] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 1587–1596.
- [39] A. Oga and B. O. Apduhan, "Towards a dynamic computation offloading mechanism with twin delayed DDPG in edge computing," in *Proc. Int. Conf. Comput. Sci. Appl.*, Springer, 2023, pp. 133–149.
- [40] K. Zheng, G. Jiang, X. Liu, K. Chi, X. Yao, and J. Liu, "DRL-based offloading for computation delay minimization in wireless-powered multi-access edge computing," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1755–1770, Mar. 2023.
- [41] H. Wang, Y. Bai, and X. Xie, "Deep reinforcement learning based resource allocation in delay-tolerance-aware 5G industrial IoT systems," *IEEE Trans. Commun.*, vol. 72, no. 1, pp. 209–221, Jan. 2024.



Renchao Xie (Senior Member, IEEE) received the PhD degree in electrical engineering from the Beijing University of Posts and Telecommunications, in 2012. From 2010 to 2011, he visited Carleton University, Ottawa, ON, Canada, as a visiting scholar. Currently, he is a professor with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications. His research interests include edge computing, computing power networks, industrial Internet of Things. He has published 80+ papers in reputable journals/conferences, five books, and 41 granted patents. He received the Best Paper Awards at IEEE ICC 2014 and ICC 2022, respectively. He has served as the Technical Program Committee (TPC) co-chair of numerous conferences. He has also served for several journals as a reviewer, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Service Computing*, *IEEE Transactions on Communications* and so on.



Tao Huang (Senior Member, IEEE) received the BS degree in communication engineering from Nankai University, Tianjin, China, in 2002, and the MS and PhD degrees in communication and information system from the Beijing University of Posts and Telecommunications, Beijing, China, in 2004 and 2007 respectively. He is currently a professor with the Beijing University of Posts and Telecommunications. His current research interests include network architecture, network artificial intelligence, routing and forwarding, and network virtualization.



Tianjiao Chen received the PhD degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022. He is currently a research fellow with China Mobile Research Institute. His research interests include machine learning, network virtualization and software defined networking (SDN).



Haoxiang Qiu received the BS degree from the Beijing University of Posts and Telecommunications, in 2023. He is currently working toward the MS degree with the Beijing University of Posts and Telecommunications, China. He current research interests include edge/cloud computing and task scheduling.



Qinqin Tang (Member, IEEE) received the PhD degree from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2022. She is currently an Associate Researcher with the School of Information and Communication Engineering, BUPT. Her current research interests include mobile edge networks, network artificial intelligence, traffic offloading, and resource management and allocation.



Zehui Xiong (Member, IEEE) received the PhD degree in computer science and engineering with Nanyang Technological University (NTU), Singapore. He is currently a full professor with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, U.K.. Prior to that, he was with the Singapore University of Technology and Design, and Alibaba-NTU Singapore Joint Research Institute. He was a visiting scholar with Princeton University and University of Waterloo. His research interests include wireless networks, Internet of Things, edge intelligence, semantic communications, generative AI, and Metaverse. Recognized as a Highly Cited Researcher, he has published more than 250 peer-reviewed research papers in leading journals, with numerous Best Paper Awards from international flagship conferences. He serves as an editor or guest editor for many leading journals. Featured in Forbes Asia 30U30, he serves as the chair for numerous international conferences. His honors include the IEEE Asia Pacific Outstanding Young Researcher Award, IEEE Early Career Award for Excellence in Scalable Computing, IEEE Technical Committee on Blockchain and Distributed Ledger Technologies Early Career Award, IEEE Internet Technical Committee Early Achievement Award, IEEE TCSVC Rising Star Award, IEEE TCI Rising Star Award, IEEE TCCLD Rising Star Award, IEEE ComSoc Outstanding Paper Award, IEEE Best Land Transport Paper Award, IEEE Asia Pacific Outstanding Paper Award, IEEE CSIM Technical Committee Best Journal Paper Award, IEEE SPCC Technical Committee Best Paper Award, IEEE Big Data Best Influential Conference Paper Award, and IEEE VTS Singapore Best Paper Award.



Li Feng (Member, IEEE) received the BS degree from the Beijing University of Posts and Telecommunications, in 2022. She is currently working toward the PhD degree with the Beijing University of Posts and Telecommunications, China. Her current research interests include edge/cloud computing and task scheduling.